MANIPAL
ACADEMY of HIGHER EDUCATION
(Institution of Eminence Deemed to be University)

T. A. PAI MANAGEMENT INSTITUTE (TAPMI)

BBAH – Semester 5

INTRODUCTION TO PYTHON PROGRAMMING

Semester Project Guidelines

Course Code: BBAT5E07

Project Component: Group Project Presentation

Academic Session: 2026–27

Faculty: Prof. Jinu Sara

1. Project Overview

The semester project is the major applied component of Introduction to Python Programming. It is designed to assess whether students can move beyond isolated Python exercises and apply programming and computational thinking to a meaningful business context.

The project should demonstrate the complete chain:

BUSINESS PROBLEM → PROBLEM UNDERSTANDING → COMPUTATIONAL / DESIGN THINKING → DATA & REQUIREMENTS → PYTHON SOLUTION → ANALYSIS / VISUALISATION → BUSINESS INSIGHT → RECOMMENDATION

The final submission and presentation should make this chain visible and logically traceable.

1.1 Alignment with the Course

The course outline describes the course as a beginner-oriented Python course for management students, with emphasis on practical business problems, data handling, basic data analysis, and visualisation. The project therefore expects students to demonstrate practical application rather than programming for its own sake. The course learning outcomes are CLO1: Fundamental Python programming concepts; CLO2: Python programs for simple business problems; and CLO3: Python libraries for real-world problem-solving, basic data analysis, and data visualisation. The project component is a Group Project Presentation mapped to CLO1–3.

1.2 What the Project Must Achieve

- Identify a real and clearly bounded problem in a specific business/domain context.
- Justify why the problem matters using evidence from the internship, organisation, industry, domain literature, observations, interviews, surveys, or credible secondary data.
- Define the problem in a precise and solution-oriented problem statement.
- Apply a systematic methodology: Computational Thinking and/or Design Thinking.
- Collect, create, clean, structure, or otherwise prepare appropriate data for the problem.
- Develop a functioning Python-based solution, analysis, prototype, or decision-support application.
- Demonstrate meaningful use of the Python concepts covered in the course.
- Use appropriate Python libraries for numerical processing, data analysis, and visualisation where relevant.
- Interpret the computational output in business terms rather than merely displaying code or graphs.
- Present limitations, assumptions, ethical considerations, and possible future improvements.

2. Project Eligibility and Scope

2.1 Domain Specificity – Mandatory

Each project must belong to a clearly identified domain. Suitable domains include, but are not limited to:

Domain / Elective Area	Illustrative Problem Directions
Marketing	Customer segmentation, campaign performance, lead conversion, product preference, churn indicators
Finance	Portfolio analysis, expense classification, profitability analysis, financial trend analysis, credit-risk indicators
HR	Employee attrition indicators, recruitment funnel analysis, attendance/workforce analytics, training effectiveness
Operations / Supply Chain	Inventory optimisation, demand patterns, delivery performance, supplier analysis
Retail	Purchase analysis, basket patterns, customer value, product performance, inventory movement

Business Analytics	KPI dashboards, descriptive analysis, trend detection, classification or decision-support prototypes
Entrepreneurship / Start-up	Customer discovery analysis, pricing analysis, sales forecasting support, operational tracking
Other Elective / Internship Domain	A problem directly connected to the student's area of study or internship experience

2.2 Internship / Elective Connection

At least one of the following connections should be clearly established:

- The problem was directly observed during an internship.
- The problem is based on a real process, dataset, workflow, or decision encountered during your internship.
- The problem directly relates to the student's elective subject/domain and uses concepts from that domain.
- The project addresses a realistic problem faced by a business, institution, customer group, process, or market segment.

Projects based solely on generic tutorial datasets or generic applications (for example, a calculator, basic quiz, simple billing program, or unrelated game) will not meet the intended project standard unless they are substantially reframed around a genuine domain-specific problem with evidence and analysis.

3. Problem Identification and Problem Statement

3.1 The Problem Statement Must Answer

- What is the business/domain problem?
- Who is affected by the problem?
- Where and in what context does it occur?
- What evidence shows that the problem exists?
- Why is the problem important?
- What decision, process, cost, risk, inefficiency, opportunity, or customer issue is involved?
- What can Python realistically help the organisation/user understand, automate, analyse, predict, or decide?
- What is the intended outcome of the proposed solution?

3.2 Weak vs Strong Problem Statements

Weak	Strong
“We want to analyse sales using Python.”	“The retail outlet has monthly transaction data but currently lacks a systematic method to identify high-value customer segments and declining product categories. This project develops a Python-based analysis to identify these patterns and support targeted sales decisions.”
“We will make an HR project using employee data.”	“The organisation experiences inconsistent employee attrition monitoring. Using historical employee records, the project will identify patterns associated with attrition and create a management-oriented dashboard of relevant indicators.”
“We will create a stock market program.”	“The project investigates how a student's investment portfolio is distributed across selected stocks and uses Python to calculate exposure, compare performance, visualise allocation, and support portfolio review.”

4. Required Problem-Solving Methodology

Students must explicitly state and demonstrate the methodology used. The preferred approaches are Computational Thinking and/or Design Thinking. A project should not merely mention the methodology; the stages must be visible in the project work.

4.1 Option A – Computational Thinking

Stage	Expected Project Evidence
Decomposition	Break the broad business problem into smaller, manageable sub-problems or tasks.
Pattern Recognition	Identify recurring trends, relationships, similarities, anomalies, or useful structures in the problem/data.
Abstraction	Identify the essential variables, entities, rules, assumptions, and information required; remove irrelevant detail.
Algorithm Design	Develop a logical, ordered sequence of steps for the proposed computational solution before coding.

4.2 Option B – Design Thinking

Stage	Expected Project Evidence
Empathise	Understand users/stakeholders through observation, interviews, surveys, internship experience, or secondary evidence.
Define	Convert findings into a focused and evidence-based problem statement.
Ideate	Consider alternative solutions and justify the selected approach.
Prototype	Develop the Python solution/prototype, analysis workflow, dashboard, or decision-support tool.
Test	Evaluate outputs with sample/real data, users, scenarios, validation checks, or comparison against expected results.

4.3 Methodology Deliverables

- A visual methodology flow diagram.
- Clear explanation of each stage applied to the chosen problem.
- Evidence showing how the methodology influenced the final Python solution.
- A traceable link between the problem, requirements, algorithm, code, results, and business recommendation.

5. Mandatory Project Report Structure

The final report should follow the structure below. Students may add sub-sections where appropriate.

Section	Required Content
1. Title Page	Project title, domain/elective, student names/IDs, batch, course, faculty, date.
2. Abstract / Executive Summary	Problem, data/context, methodology, Python solution, key findings, and business implication in concise form.
3. Introduction	Domain background, organisation/context, motivation, business relevance, scope.
4. Problem Statement & Justification	Precise problem statement, evidence, stakeholders, impact, need for solution, project objectives.
5. Existing Practice / Related Solutions	Current process, existing tools/products/approaches, strengths, gaps, and opportunity for the proposed solution.

6. Methodology	Computational Thinking and/or Design Thinking stages, data collection approach, workflow, assumptions, and project plan.
7. Data Description & Preparation	Data sources, variables, structure, data quality, cleaning/transformation, privacy/ethics, and limitations.
8. Solution Design	System/process flow, algorithm/pseudocode, input-process-output model, modules/functions, data structures, and design decisions.
9. Python Implementation	Annotated explanation of the Python concepts, functions, libraries, and major code components used.
10. Results & Analysis	Outputs, calculations, tables, visualisations, findings, validation/testing, and interpretation.
11. Business Interpretation & Recommendations	What the results mean, decisions supported, actionable recommendations, expected value/impact.
12. Limitations & Future Scope	Technical, data, methodological, operational, and ethical limitations; realistic improvements.
13. Conclusion	How the project addressed the original problem and what was achieved.
14. References	All academic, industry, organisational, dataset, library, and web sources used.
15. Appendix	Selected/full code, sample input/output, data dictionary, questionnaire/interview guide, screenshots, testing evidence, etc.

6. Python Implementation Requirements

The project is an assessment of the learning achieved across the Introduction to Python Programming course. Students must demonstrate that the Python concepts are being used purposefully within the project rather than inserted merely to satisfy a checklist. The course covers the progression from fundamentals through data structures, files, object-oriented concepts, plots, algorithms, NumPy, Pandas, Matplotlib and Seaborn.

6.1 Concepts Expected to be Demonstrated

Course Topic	Project Evidence / Possible Application
Introduction to Python	Python environment, scripts, syntax, comments, variables, identifiers, keywords, basic input/output.
Numbers & Strings	Numerical calculations, string handling, formatting, conversion, business metrics.
Decisions	Business rules, thresholds, categories, eligibility, risk bands, conditional decisions.
Loops	Repeated processing of records, transactions, customers, products, or scenarios.
Functions	Modular/reusable calculations and logically separated components.
Lists	Collections of records, values, categories, transactions, or intermediate results.
Files & Exceptions	Reading/writing files and handling missing/invalid input or file-related errors.
Sets & Dictionaries	Unique categories/entities, lookup structures, key-value records, aggregation.
Objects, Classes & Inheritance	Where suitable, represent business entities or reusable application components; inheritance may be demonstrated where it has a genuine design rationale.
Basic Plots	Simple visual representation of business variables and trends.
Recursion	A small, meaningful recursive component where the problem structure justifies it; otherwise document why recursion is not appropriate and demonstrate the concept in a controlled supporting example.
Sorting & Searching	Rank customers/products, find records, identify top/bottom performers, efficient lookup logic.
NumPy	Numerical arrays, vectorised calculations, summary statistics or numerical processing.

Pandas	DataFrame-based data loading, cleaning, filtering, grouping, aggregation and analysis.
Matplotlib & Seaborn	Business charts and statistical visualisations with appropriate labels, titles and interpretation.

IMPORTANT
Every concept used must have a reason for being present. Students should be able to explain, during the presentation/viva, what the concept does, why it was selected, and how it contributes to the solution. The course session plan explicitly includes all of the above topics.

6.2 Libraries

Students should select libraries according to project requirements. At minimum, projects involving structured business data should consider NumPy and Pandas, while projects requiring visualisation should use Matplotlib and/or Seaborn.

- NumPy – numerical arrays and numerical computation.
- Pandas – structured data manipulation and analysis.
- Matplotlib – general-purpose visualisation.
- Seaborn – statistical and business-oriented visualisation.
- Other libraries may be used only when they add clear value and students can explain their use.

7. Data and Evidence Requirements

- Use real organisational/internship data where permission is available, or an appropriately anonymised dataset.
- If real organisational data cannot be used, use a credible public dataset or construct a clearly documented simulated dataset based on a real business process.
- Do not fabricate evidence and present it as real organisational data.
- Explain every important variable used in the analysis.
- State data source, time period, sample size, units, assumptions, and known limitations.
- Remove or anonymise confidential/personal information.
- Explain any data cleaning, missing-value treatment, filtering, transformation, encoding, or aggregation performed.

8. Results, Visualisation and Business Interpretation

A technically correct program is not sufficient. The project must explain what the results mean for the business/domain.

- Present important outputs clearly.
- Use appropriate charts rather than unnecessary visualisations.
- Label axes, units, titles, and legends where required.
- Identify meaningful trends, differences, relationships, anomalies, or patterns.
- Translate technical findings into business language.
- Connect every major finding to a decision or recommendation.
- Avoid claiming causation when the analysis only demonstrates association or pattern.

9. Academic Integrity, Ethics and Responsible Data Use

- All code submitted must be understood by the team members who present it.
- Students must disclose external code, libraries, tutorials, templates, datasets, and AI-assisted development where applicable, according to institutional requirements.
- Do not submit copied projects or substantially reproduce another team's work.
- Do not expose confidential internship information, customer identifiers, financial information, employee information, or proprietary business data.
- Use anonymisation or aggregation when necessary.
- Clearly distinguish real data, simulated data, assumptions, and illustrative examples.
- Recommendations should acknowledge relevant business, privacy, fairness, or ethical implications.

10. Group Project Expectations

- Every member must understand the complete project at a level sufficient to answer questions.
- Roles may be divided (domain research, data preparation, coding, analysis, visualisation, documentation), but knowledge must not remain isolated.
- The contribution of each member should be documented.
- The final presentation should be coherent and presented as one integrated solution.

11. Expected Deliverables

Deliverable	Minimum Expectation
Project Proposal	Title, domain, problem statement, justification, objectives, data source, methodology, preliminary solution idea.
Final Report	Complete structured report following Section 5.
Python Source Code	Clean, readable, commented, modular code with required libraries and reproducible execution.
Dataset / Data Evidence	Dataset or anonymised sample plus data dictionary and source/assumption statement.
Methodology Diagram	Computational Thinking and/or Design Thinking process applied to the project.
Results	Outputs, analysis, charts, validation/testing evidence, and business interpretation.
Presentation	Concise professional presentation demonstrating problem → methodology → Python solution → findings → recommendations.
Appendix	Supporting code, screenshots, test cases, questionnaires, sample outputs, etc.

12. Presentation Guidelines

The presentation should be structured around the problem and solution—not around a sequence of Python commands.

Recommended Slide	Content
1	Project title, team, domain
2	Domain / internship context
3	Problem and evidence
4	Problem statement and objectives

5	Computational Thinking / Design Thinking methodology
6	Data and variables
7	Solution architecture / algorithm / IPO
8	Python concepts and libraries used
9	Key implementation / screenshots
10	Results and visualisations
11	Business insights
12	Recommendations and expected impact
13	Limitations and future scope
14	Conclusion
15	Q&A / contribution summary

Evaluation principle: The number of lines of code is not a measure of project quality. Marks will reflect the appropriateness, correctness, integration and business value of the Python solution.

Important: Students are expected to demonstrate meaningful application of the Python concepts covered in the course. Concepts should not be artificially inserted merely to satisfy a checklist. Where a topic such as inheritance or recursion is not naturally required by the chosen problem, students should be able to explain why it is unsuitable for the core solution rather than adding unnecessary complexity. However, the overall project should demonstrate sufficient breadth across the course.

All the Best!!!